

UNIVERSIDADE DE MOGI DAS CRUZES
BACHARELADO EM ENGENHARIA DE SOFTWARE

DIEGO ALVES DA SILVA FAGUNDES

SISTEMA DE AUTOMAÇÃO RESIDENCIAL IOT COM ESP32

Mogi das Cruzes

2026

DIEGO ALVES DA SILVA FAGUNDES

SISTEMA DE AUTOMAÇÃO RESIDENCIAL IOT COM ESP32

Trabalho apresentado para o Prof. Alessandro Aparecido da Silva sobre atividade relacionada à disciplina de IOT – Internet das coisas do curso de Bacharelado em Engenharia de Software.

Mogi das Cruzes

2026

RESUMO

Este projeto descreve o desenvolvimento e a simulação de um sistema de automação residencial (IoT) focado em um "quarto inteligente", utilizando o microcontrolador ESP32 programado em MicroPython. A arquitetura contempla a leitura de quatro sensores distintos (temperatura/umidade, luminosidade, presença e abertura de porta) e o controle independente de quatro atuadores (relé para iluminação, anel NeoPixel para ar-condicionado, servomotor para persiana e buzzer para alarme), além de um painel OLED para monitoramento local. A comunicação entre o hardware simulado na plataforma Wokwi e a interface do usuário é realizada bidirecionalmente via protocolo MQTT utilizando o broker público HiveMQ. Para a visualização e controle remoto, foi implementada uma dashboard interativa no Node-RED e integra serviços em nuvem, realizando o registro de dados em uma planilha do Google Sheets e o envio de alertas de segurança por e-mail (SMTP) ao proprietário.

Palavras-Chaves: IoT; ESP32; MicroPython; MQTT, Node-RED.

ABSTRACT

This project describes the development and simulation of a home automation (IoT) system focused on a "smart room," using the ESP32 microcontroller programmed in MicroPython. The architecture includes the reading of four distinct sensors (temperature/humidity, luminosity, presence, and door opening) and the independent control of four actuators (relay for lighting, NeoPixel ring for air conditioning, servomotor for blinds, and buzzer for alarm), in addition to an OLED panel for local monitoring. Communication between the simulated hardware on the Wokwi platform and the user interface is performed bidirectionally via the MQTT protocol using the HiveMQ public broker. For visualization and remote control, an interactive dashboard was implemented in Node-RED and integrates cloud services, recording data in a Google Sheets spreadsheet and sending security alerts via email (SMTP) to the owner.

Keywords: IoT; ESP32; MicroPython; MQTT, Node-RED.

Sumário

1. INTRODUÇÃO	6
2. OBJETIVOS	7
2.1. Objetivo Geral	7
2.2. Objetivos Específicos.....	7
3. MATERIAIS E MÉTODOS.....	8
3.1. Arquitetura de Hardware (Sensores e Atuadores)	8
3.2. Software e Protocolos	8
3.3. Integração e Comunicação (MQTT e Web Server).....	9
4. DESENVOLVIMENTO DO PROJETO.....	10
4.1. Lógica de Leitura e Acionamento (MicroPython)	10
4.2. Dashboard no Node-RED	10
4.3. Notificações e Armazenamento (Google Sheets e E-mail)	10
5. DEMONSTRAÇÃO E RESULTADOS	12
6. CONCLUSÃO	13
REFERÊNCIAS.....	14

1. INTRODUÇÃO

A Internet das Coisas (IoT) transformou a maneira como interagimos com os ambientes, permitindo o monitoramento e o controle remoto de dispositivos físicos. No contexto da automação residencial, a integração de sensores e atuadores proporciona maior conforto, eficiência energética e segurança aos usuários.

Este projeto apresenta a implementação completa de um sistema IoT para automação de um quarto inteligente. O ambiente de desenvolvimento baseia-se no simulador Wokwi integrado ao Visual Studio Code, utilizando o microcontrolador ESP32 como núcleo de processamento. A programação foi desenvolvida em linguagem MicroPython, garantindo a leitura constante das variáveis físicas do ambiente e o controle preciso dos dispositivos. O uso do protocolo MQTT e do Node-RED assegura a comunicação rápida e confiável entre o hardware e o usuário.

2. OBJETIVOS

2.1. OBJETIVO GERAL

Projetar e implementar um sistema IoT completo para controle e automação residencial, validando o funcionamento via simulador Wokwi, utilizando a arquitetura ESP32, programação MicroPython, comunicação MQTT e interface gráfica no Node-RED.

2.2. OBJETIVOS ESPECÍFICOS

- Configurar o ESP32 para ler dados de quatro sensores distintos.
- Controlar no mínimo quatro atuadores de forma independente, com respostas locais e remotas.
- Prover um painel OLED local para demonstração visual da situação do ambiente.
- Implementar comunicação bidirecional utilizando o protocolo MQTT via broker HiveMQ.
- Desenvolver um dashboard no Node-RED para visualização e acionamento remoto em tempo real.
- Implementar lógica de salvamento de dados via Webhook em uma planilha do Google Sheets.
- Desenvolver um sistema de alertas por e-mail notificando o proprietário em caso de eventos específicos de segurança.

3. MATERIAIS E MÉTODOS

3.1. ARQUITETURA DE HARDWARE (SENSORES E ATUADORES)

O hardware simulado no Wokwi foi estruturado para representar as necessidades de um quarto inteligente.

Sensores Utilizados:

- DHT22: Medição de temperatura do ambiente, usado como gatilho para o ar-condicionado.
- LDR (Resistor Dependente de Luz): Conectado via pino analógico (ADC) para monitorar a luminosidade natural e determinar a abertura ou fechamento da persiana.
- PIR (Sensor de Presença): Capaz de detectar movimento no ambiente, acionando a iluminação e/ou gerando alertas de segurança.
- Slide-switch (Sensor Magnético de Porta): Simula a abertura ou fechamento da porta/portão principal.

Atuadores e Monitores Utilizados:

- Relé (Luz do Quarto): Simula o chaveamento de uma lâmpada comum.
- NeoPixel LED Ring (Ar-condicionado): Atua como representação visual do estado do ar-condicionado (acionado ou desligado).
- Servomotor (Persiana): Varia sua angulação (Duty Cycle via PWM) para abrir ou fechar as persianas.
- Buzzer (Alarme): Emite som quando o modo de segurança detecta intrusões ou violação de porta.
- Display OLED (SSD1306): Comunicação via I2C, exibindo o status em tempo real da rede Wi-Fi, valores de temperatura, luminosidade e estado da porta e presença.

3.2. SOFTWARE E PROTOCOLOS

- Ambiente de Desenvolvimento: Visual Studio Code com a extensão Wokwi Simulator para prototipagem do hardware e execução do código.
- Linguagem: MicroPython, compatível com a arquitetura ESP32.
- Comunicação MQTT: Utilizada a biblioteca umqtt.simple conectada ao broker público broker.hivemq.com.

- Interface Gráfica (Node-RED): Fluxo construído para interceptar mensagens MQTT e apresentá-las utilizando a paleta node-red-dashboard.

3.3. INTEGRAÇÃO E COMUNICAÇÃO (MQTT E WEB SERVER)

A arquitetura de rede do sistema é híbrida. O ESP32 conecta-se ao Wi-Fi virtual (Wokwi-GUEST) e estabelece duas frentes de controle:

- Servidor Socket Local: O ESP32 hospeda um servidor na porta 80 que intercepta parâmetros HTTP (ex: `/?luz=on`, `/?persiana=off`), permitindo o controle via requisições na rede local.
- Broker MQTT: O ESP32 publica os estados dos sensores no tópico `diego/quarto/sensores` num formato JSON estruturado e é inscrito (subscribe) no tópico `diego/quarto/comandos` para receber strings de comando (ex: `LUZ_ON`, `ALARME_OFF`) enviadas pela dashboard.

4. DESENVOLVIMENTO DO PROJETO

4.1. LÓGICA DE LEITURA E ACIONAMENTO (MICROPYTHON)

O arquivo `main.py` foi estruturado de forma não bloqueante. Foi configurado um loop contínuo (`while True`) que executa tarefas baseadas em deltas de tempo (utilizando `time.ticks_ms()`). A cada 2 segundos, o firmware faz a varredura do DHT22, LDR, PIR e Slide-switch.

Se um comando de acionamento for recebido via MQTT (função `callback_mqtt`) ou via Web (servidor socket HTTP), o sistema atualiza variáveis globais de estado (ex: `estado_luz`, `estado_persiana`) e chama a rotina `atualiza_atuadores()`, que reflete os dados nos pinos físicos (GPIOs, PWM e I2C).

4.2. DASHBOARD NO NODE-RED

O fluxo do Node-RED (exportado no arquivo `flows.json`) foi construído para fornecer uma interface gráfica.

- **Leitura:** Um nó MQTT-in recebe o JSON do tópico `diego/quarto/sensores`. Um nó Function processa os dados, separando-os para alimentar os medidores (Gauge de temperatura) e indicadores de texto (Luminosidade, Movimento e Portão).
- **Controle:** Foram implementados botões individuais para cada atuador ("Ligar Luz", "Desligar Luz", "Abrir Persiana", "Ativar Alarme", etc.). Cada botão, ao ser pressionado, dispara um payload de string específica via um nó MQTT-out para o tópico `diego/quarto/comandos`.

4.3. NOTIFICAÇÕES E ARMAZENAMENTO (GOOGLE SHEETS E E-MAIL)

Foram estabelecidos critérios de segurança: caso o sensor PIR detecte movimento ou o sensor da porta indique abertura, um evento de alerta é acionado (com travas anti-spam para não enviar múltiplas mensagens).

A rotina `envia_alerta()` executa duas ações:

- **Google Sheets:** Realiza um HTTP POST (`urequests`) enviando um payload JSON estruturado com os dados dos sensores e o alerta disparado para o endpoint (Web App) do Google Apps Script.
- **E-mail:** Utiliza a biblioteca `usocket` e `ssl` para conectar diretamente aos servidores SMTP do Gmail (porta 465). Após autenticação segura utilizando

senha de aplicativo, envia um e-mail com cabeçalho completo ao proprietário detalhando os alertas e os status em tempo real do quarto inteligente.

5. DEMONSTRAÇÃO E RESULTADOS

Os testes de integração comprovaram o funcionamento de todos os requisitos.

- Simulação no Wokwi: O hardware virtual refletiu com perfeição a ligação dos cabos, com o Display OLED mostrando os valores e os LEDs/Servos respondendo em milissegundos.
- Comunicação Node-RED: O Dashboard operou em tempo real, registrando variações do DHT22 simulado no Wokwi e enviando os comandos de acionamento com êxito. O caráter bidirecional (ação no botão do Node-RED resultando em ativação do relé no Wokwi) foi validado.
- Avisos em Nuvem: O Webhook da planilha recebeu os registros (conforme documentado na imagem planilha.png do projeto original) e a caixa de entrada do e-mail do proprietário recebeu o aviso detalhado formatado (conforme imagem email.png do repositório).

Todo o funcionamento supracitado encontra-se documentado, também, em vídeo gravado demonstrando as ativações e integrando a explicação da lógica de funcionamento de ponta a ponta.

6. CONCLUSÃO

A implementação do projeto concluiu todos os objetivos propostos. Foi desenvolvido um sistema robusto de IoT capaz de comunicar o MicroPython em um ESP32 simulado no Wokwi com plataformas externas modernas (Node-RED e Google Cloud). A exigência das comunicações de protocolos diferenciados (HTTP, MQTT, SMTP, I2C, PWM) proporcionou um entendimento aprofundado na criação de soluções para Automação Residencial e desenvolvimento escalável, unindo controle autônomo (sensor-atuador), monitoramento local (OLED) e supervisão remoto (Node-RED).

REFERÊNCIAS

ESPRESSIF SYSTEMS. ESP32 Technical Reference Manual. 2026.

MICROPYTHON. MicroPython Documentation. Disponível em:

<https://docs.micropython.org/>. Acesso em: 2026.

MQTT.ORG. MQTT Version 3.1.1 Specification. Disponível em: <https://mqtt.org/>.

Acesso em: 2026.

NODE-RED. Node-RED Documentation. Disponível em: <https://nodered.org/docs/>.

Acesso em: 2026.

WOKWI. Wokwi Simulator Documentation. Disponível em: <https://docs.wokwi.com/>.

Acesso em: 2026.